

CLASE WORD TOOLKIT

VERSION 1 (NO OPTIMIZADA Y USO DE SORT):

```
#include "word_toolkit.hpp"
#include <algorithm>

/* Pre: Cert
Post: Retorna cert si i només si les lletres de l'string s estan
en ordre lexicogràfic ascendent. */
bool word_toolkit::es_canonic(const string& s) throw() { // cost  $\Theta(n)$ 
    bool canonic = true;
    for (unsigned int i = 1; i < s.size(); i++) {
        if (s[i] < s[i - 1]) {
            canonic = false;
            break;
        }
    }
    return canonic;
}

/* Pre: Cert
Post: Retorna l'anagrama canònic de l'string s.
Veure la classe anagrames per saber la definició d'anagrama canònic. */
string word_toolkit::anagrama_canonic(const string& s) throw() { //
cost  $\Theta(n \log n)$ 
    string letras_validas;
    for (unsigned int i = 0; i < s.size(); i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') { // Solo incluye letras entre
'A' y 'Z'
            letras_validas += s[i];
        }
    }
    sort(letras_validas.begin(), letras_validas.end()); // Ordena
lexicogràficament
    return letras_validas;
}

/* Pre: L és una llista no buida de paraules formades exclusivament
amb lletres majúscules de la 'A' a la 'Z' (excloses la 'Ñ', 'Ç',
majúscules accentuades, ...).
Post: Retorna el caràcter que no apareix a l'string excl i és
el més freqüent en la llista de paraules L.
```

```

En cas d'empat, es retornaria el caràcter alfabèticament menor.
Si l'string excl inclou totes les lletres de la 'A' a la 'Z' es
retorna el caràcter '\0', és a dir, el caràcter de codi ASCII 0. */
char word_toolkit::mes_frequent(const string& excl, const list<string>&
L) throw() { // cost  $\Theta(n + m \cdot k)$ 
    // Contadores para las frecuencias
    int max_frecuencia = 0;
    char letra_mas_frecuente = '\0';

    // Probar cada letra del alfabeto
    for (char letra = 'A'; letra <= 'Z'; ++letra) {
        // Comprobar si la letra está en el string excl (no la contamos
si está excluida)
        bool excluida = false;
        for (unsigned int i = 0; i < excl.size(); ++i) {
            if (excl[i] == letra) {
                excluida = true;
                break; // Salimos del bucle si encontramos la letra en
excl
            }
        }

        // Si la letra está excluida, no la contamos
        if (!excluida) {
            // Contar cuántas veces aparece la letra en las palabras de
L

            int frecuencia_actual = 0;
            for (list<string>::const_iterator it = L.begin(); it !=
L.end(); ++it) {
                const std::string& palabra = *it;
                for (unsigned int j = 0; j < palabra.size(); ++j) {
                    if (palabra[j] == letra) {
                        ++frecuencia_actual;
                    }
                }
            }

            // Actualizar si es más frecuente o alfabéticamente menor en
caso de empate
            if (frecuencia_actual > max_frecuencia or (frecuencia_actual
== max_frecuencia and letra < letra_mas_frecuente)) {
                max_frecuencia = frecuencia_actual;
                letra_mas_frecuente = letra;
            }
        }
    }
}

```

```

    }

    }

}

return letra_mas_frecuente;
}

```

VERSION 2 (OPTIMIZADA):

```

#include "word_toolkit.hpp"

/* Pre: Cert
   Post: Retorna cert si i només si les lletres de l'string s estan
   en ordre lexicogràfic ascendent. */
bool word_toolkit::es_canonic(const string& s) throw() { // cost  $\Theta(n)$ 
    for (unsigned int i = 1; i < s.size(); i++) {
        if (s[i] < s[i - 1]) { // Compara cada lletra amb l'anterior per
assegurar-se que l'ordre és ascendent
            return false;
        }
    }
    return true;
}

/* Pre: Cert
   Post: Ordena el string s lexicogràficament (alfabèticament ascendent).
   */
void ordenar_lexicograficament(string& paraula) { // cost  $\Theta(n)$ 
    int comptador[26] = {0}; // Array per comptar la freqüència de cada
lletra majúscula

    // Comptem la freqüència de cada lletra
    for (unsigned int i = 0; i < paraula.size(); ++i) {
        if (paraula[i] >= 'A' and paraula[i] <= 'Z') {
            ++comptador[paraula[i] - 'A'];
        }
    }

    unsigned int index = 0;
    // Reconstruïm el string ordenat basant-nos en la freqüència de cada
lletra
    for (int i = 0; i < 26; ++i) {
        while (comptador[i] > 0) {

```

```

        paraula[index++] = 'A' + i;
        --comptador[i];
    }
}

    paraula = paraula.substr(0, index); // Retallem el string fins a
l'última lletra vàlida
}

/* Pre: Cert
Post: Retorna l'anagrama canònic de l'string s.
Veure la classe anagrames per saber la definició d'anagrama canònic.
*/
string word_toolkit::anagrama_canonic(const string& s) throw() { //
cost  $\Theta(n)$ 
    string lletres_valides;
    // Filtra només les lletres majúscules de la 'A' a la 'Z'
    for (unsigned int i = 0; i < s.size(); i++) {
        if (s[i] >= 'A' and s[i] <= 'Z') {
            lletres_valides += s[i];
        }
    }
    ordenar_lexicograficament(lletres_valides); // Ordena
lexicogràficament les lletres vàlides
    return lletres_valides;
}

/* Pre: L és una llista no buida de paraules formades exclusivament
amb lletres majúscules de la 'A' a la 'Z' (excloses la 'Ñ', 'Ç',
majúscules accentuades, ...).
Post: Retorna el caràcter que no apareix a l'string excl i és
el més freqüent en la llista de paraules L.
En cas d'empat, es retornaria el caràcter alfabèticament menor.
Si l'string excl inclou totes les lletres de la 'A' a la 'Z' es
retorna el caràcter '\0', és a dir, el caràcter de codi ASCII 0. */
char word_toolkit::mes_frequent(const string& excl, const list<string>&
L) throw() { // cost  $\Theta(n + m * k + 26) \approx \Theta(n + m * k)$ 
    bool exclosa[26] = {false};
    // Marca les lletres a excl com a excloses
    for (unsigned int i = 0; i < excl.size(); ++i) {
        if (excl[i] >= 'A' and excl[i] <= 'Z') {
            exclosa[excl[i] - 'A'] = true;
        }
    }

```

```

}

int comptador[26] = {0};

// Recorre cada paraula de la llista i compta la freqüència de les
lletres no excloses
for (list<string>::const_iterator it = L.begin(); it != L.end();
++it) {
    const string& paraula = *it;
    for (unsigned int j = 0; j < paraula.size(); ++j) {
        char lletra = paraula[j];
        if (lletra >= 'A' and lletra <= 'Z' and !exclosa[lletra -
'A']) {
            ++comptador[lletra - 'A'];
        }
    }
}

char lletra_mes_frequent = '\0';
int max_frequencia = 0;
// Cerca la lletra més freqüent que no estigui exclosa
for (char lletra = 'A'; lletra <= 'Z'; ++lletra) {
    if (comptador[lletra - 'A'] > max_frequencia) {
        max_frequencia = comptador[lletra - 'A'];
        lletra_mes_frequent = lletra;
    }
}

return lletra_mes_frequent; // Retorna la lletra més freqüent o '\0'
si cap lletra és vàlida
}

```

CLASE ITER SUBSET

VERSION 1 (Violación de segmento):

```
#include "iter_subset.hpp"

/* Pre: Cert
Post: Construeix un iterador sobre els subconjunts de k elements
de {1, ..., n}; si k > n no hi ha res a recórrer. */
iter_subset::iter_subset(nat n, nat k) throw(error) { // cost  $\Theta(k)$ 
    _n = n;
    _k = k;
    _end = false;
    if (k > n) {
        _end = true; // No hay subconjuntos posibles
    } else {
        _current.resize(k);
        for (nat i = 0; i < k; ++i) {
            _current[i] = i + 1; // Inicializar el primer subconjunto
        }
    }
}

/* Tres grans. Constructor per còpia, operador d'assignació i
destructor. */
iter_subset::iter_subset(const iter_subset& its) throw(error) { // cost
 $\Theta(k)$ 
    _n = its._n;
    _k = its._k;
    _current = its._current;
    _end = its._end;
}

iter_subset& iter_subset::operator=(const iter_subset& its)
throw(error) { // cost  $\Theta(k)$ 
    if (this != &its) {
        _n = its._n;
        _k = its._k;
        _current = its._current;
        _end = its._end;
    }
    return *this;
}
```

```

iter_subset::~~iter_subset() throw() { // No necesitamos hacer nada
especial aquí

    // No hace falta liberar memoria explícitamente, ya que std::vector
maneja la memoria
}

/* Pre: Cert
Post: Retorna cert si l'iterador ja ha visitat tots els subconjunts
de k elements presos d'entre n; o dit d'una altra forma, retorna
cert quan l'iterador apunta a un subconjunt sentinella fictici
que queda a continuació de l'últim subconjunt vàlid. */
bool iter_subset::end() const throw() { // cost  $\Theta(1)$ 
    return _end;
}

/* Operador de desreferència.
Pre: Cert
Post: Retorna el subconjunt apuntat per l'iterador;
llança un error si l'iterador apunta al sentinella. */
subset iter_subset::operator*() const throw(error) { // cost  $\Theta(1)$ 
    if (_end) {
        throw error(IterSubsetIncorr);
    }
    return _current;
}

/* Operador de preincremento.
Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i el
retorna;
no es produeix l'avançament si l'iterador ja apuntava al sentinella. */
iter_subset& iter_subset::operator++() throw() {
    if (_end) {
        return *this;
    }

    // Avanzar al siguiente subconjunto lexicográfico
    nat i = _k - 1;
    while (i >= 0 && _current[i] == _n - _k + i + 1) {
        --i;
    }

```

```

    if (i < 0) {
        _end = true; // No hay más subconjuntos
    } else {
        ++_current[i];
        for (nat j = i + 1; j < _k; ++j) {
            _current[j] = _current[j - 1] + 1;
        }
    }

    return *this;
}

/* Operador de postincremento.
Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i retorna
el seu valor
previ; no es produeix l'avançament si l'iterador ja apuntava al
sentinella. */
iter_subset iter_subset::operator++(int) throw() {
    iter_subset temp(*this);
    ++(*this);
    return temp;
}

/* Operadors relacionals. */
bool iter_subset::operator==(const iter_subset& c) const throw() { //
cost  $\Theta(k)$ 
    return _n == c._n && _k == c._k && _current == c._current && _end ==
c._end;
}

bool iter_subset::operator!=(const iter_subset& c) const throw() { //
cost  $\Theta(k)$ 
    return !(*this == c);
}

```

VERSION 2 (CANVI AL OPERATOR++() PEL SEU CORRECTE FUNCIONAMENT) :

```

#include "iter_subset.hpp"

/* Pre: Cert
Post: Construeix un iterador sobre els subconjunts de k elements

```



```

de {1, ..., n}; si k > n no hi ha res a recórrer. */
iter_subset::iter_subset(nat n, nat k) throw(error) { // cost  $\Theta(k)$ 
    _n = n;
    _k = k;
    _end = false;
    if (k > n) {
        _end = true; // No hay subconjuntos posibles
        _current.clear(); // Asegúrate de que _current esté vacío.
    } else {
        _current.resize(k);
        for (nat i = 0; i < k; ++i) {
            _current[i] = i + 1; // Inicializar el primer subconjunto
        }
    }
}

/* Tres grans. Constructor per còpia, operador d'assignació i
destructor. */
iter_subset::iter_subset(const iter_subset& its) throw(error) { // cost
 $\Theta(k)$ 
    _n = its._n;
    _k = its._k;
    _current = its._current;
    _end = its._end;
}

iter_subset& iter_subset::operator=(const iter_subset& its)
throw(error) { // cost  $\Theta(k)$ 
    if (this != &its) {
        _n = its._n;
        _k = its._k;
        _current = its._current;
        _end = its._end;
    }
    return *this;
}

iter_subset::~~iter_subset() throw() {}

/* Pre: Cert
Post: Retorna cert si l'iterador ja ha visitat tots els subconjunts
de k elements presos d'entre n; o dit d'una altra forma, retorna
cert quan l'iterador apunta a un subconjunt sentinella fictici

```

```

que queda a continuació de l'últim subconjunt vàlid. */
bool iter_subset::end() const throw() { // cost  $\Theta(1)$ 
    return _end;
}

/* Operador de desreferència.
Pre: Cert
Post: Retorna el subconjunt apuntat per l'iterador;
llança un error si l'iterador apunta al sentinella. */
subset iter_subset::operator*() const throw(error) { // cost  $\Theta(1)$ 
    if (_end) {
        throw error(IterSubsetIncorr);
    }
    return _current;
}

/* Operador de preincremento.
Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i el
retorna;
no es produeix l'avançament si l'iterador ja apuntava al sentinella. */
iter_subset& iter_subset::operator++() throw() {
    if (_end) {
        return *this; // Si ya estamos en el final, no avanzamos
    }

    // Verificamos si el subconjunto actual es el último subconjunto
posible
    subset lastSubset;
    for (nat i = _n - _k; i < _n; ++i) {
        lastSubset.push_back(i + 1); // Ajustamos los índices al
conjunto original {1, 2, ..., n}
    }

    if (_current == lastSubset) {
        _end = true; // Si el subconjunto actual es el último, marcamos
_end
    } else {
        // Generamos el siguiente subconjunto en orden lexicográfico
        for (nat i = _k - 1; i >= 0; --i) {
            if (_current[i] < _n - (_k - 1 - i)) {
                ++_current[i]; // Incrementamos el valor del elemento
actual

```

```

        // Reseteamos todos los elementos posteriores para
        formar el siguiente subconjunto
        for (nat j = i + 1; j < _k; ++j) {
            _current[j] = _current[j - 1] + 1;
        }
        return *this;
    }
}

// Si no se puede incrementar (ya es el último subconjunto),
retornamos el fin
_end = true;
}

return *this;
}

/* Operador de postincremento.
Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i retorna
el seu valor
previ; no es produeix l'avançament si l'iterador ja apuntava al
sentinella. */
iter_subset iter_subset::operator++(int) throw() {
    iter_subset temp(*this);
    if (!_end) { // si l'iterador no apunta al sentinella, avançem
        ++(*this);
    }
    return temp;
}

/* Operadors relacionals. */
bool iter_subset::operator==(const iter_subset& c) const throw() { //
cost Θ(k)
    return _n == c._n and _k == c._k and _current == c._current and _end
== c._end;
}

bool iter_subset::operator!=(const iter_subset& c) const throw() { //
cost Θ(k)
    return !(*this == c);
}

```

VERSION 3 (AÑADIT CONDICIÓN ESPECIAL OPERATOR==) :

```
#include "iter_subset.hpp"

/* Pre: Cert
Post: Construeix un iterador sobre els subconjunts de k elements
de {1, ..., n}; si k > n no hi ha res a recórrer. */
iter_subset::iter_subset(nat n, nat k) throw(error) { // cost  $\Theta(k)$ 
    _n = n;
    _k = k;
    _end = false;
    if (k > n) {
        _end = true; // No hay subconjuntos posibles
        _current.clear(); // Asegúrate de que _current esté vacío.
    } else {
        _current.resize(k);
        for (nat i = 0; i < k; ++i) {
            _current[i] = i + 1; // Inicializar el primer subconjunto
        }
    }
}

/* Tres grans. Constructor per còpia, operador d'assignació i
destructor. */
iter_subset::iter_subset(const iter_subset& its) throw(error) { // cost
 $\Theta(k)$ 
    _n = its._n;
    _k = its._k;
    _current = its._current;
    _end = its._end;
}

iter_subset& iter_subset::operator=(const iter_subset& its)
throw(error) { // cost  $\Theta(k)$ 
    if (this != &its) {
        _n = its._n;
        _k = its._k;
        _current = its._current;
        _end = its._end;
    }
    return *this;
}
```

```

iter_subset::~~iter_subset() throw() {}

/* Pre:  Cert
Post: Retorna cert si l'iterador ja ha visitat tots els subconjunts
de k elements presos d'entre n; o dit d'una altra forma, retorna
cert quan l'iterador apunta a un subconjunt sentinella fictici
que queda a continuació de l'últim subconjunt vàlid. */
bool iter_subset::end() const throw() { // cost  $\Theta(1)$ 
    return _end;
}

/* Operador de desreferència.
Pre:  Cert
Post: Retorna el subconjunt apuntat per l'iterador;
llança un error si l'iterador apunta al sentinella. */
subset iter_subset::operator*() const throw(error) { // cost  $\Theta(1)$ 
    if (_end) {
        throw error(IterSubsetIncorr);
    }
    return _current;
}

/* Operador de preincremento.
Pre:  Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i el
retorna;
no es produeix l'avançament si l'iterador ja apuntava al sentinella. */
iter_subset& iter_subset::operator++() throw() {
    if (_end) {
        return *this; // Si ya estamos en el final, no avanzamos
    }

    // Verificamos si el subconjunto actual es el último subconjunto
posible
    subset lastSubset;
    for (nat i = _n - _k; i < _n; ++i) {
        lastSubset.push_back(i + 1); // Ajustamos los índices al
conjunto original {1, 2, ..., n}
    }

    if (_current == lastSubset) {
        _end = true; // Si el subconjunto actual es el último, marcamos
_end

```

```

    } else {
        // Generamos el siguiente subconjunto en orden lexicográfico
        for (nat i = _k - 1; i >= 0; --i) {
            if (_current[i] < _n - (_k - 1 - i)) {
                ++_current[i]; // Incrementamos el valor del elemento
actual
                // Reseteamos todos los elementos posteriores para
formar el siguiente subconjunto
                for (nat j = i + 1; j < _k; ++j) {
                    _current[j] = _current[j - 1] + 1;
                }
                return *this;
            }
        }
        // Si no se puede incrementar (ya es el último subconjunto),
retornamos el fin
        _end = true;
    }

    return *this;
}

/* Operador de postincremento.
Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i retorna
el seu valor
previ; no es produeix l'avançament si l'iterador ja apuntava al
sentinella. */
iter_subset iter_subset::operator++(int) throw() {
    iter_subset temp(*this);
    if (!_end) { // si l'iterador no apunta al sentinella, avançem
        ++(*this);
    }
    return temp;
}

/* Operadors relacionals. */
bool iter_subset::operator==(const iter_subset& c) const throw() { //
cost Θ(k)
    if (_n != c._n and _k == 0 and c._k == 0 and _current == c._current
and _end == c._end) {
        return true;
    }

```

```

    }
    return _n == c._n and _k == c._k and _current == c._current and _end
    == c._end;
}

bool iter_subset::operator!=(const iter_subset& c) const throw() { //
cost  $\Theta(k)$ 
    return !(*this == c);
}

```

VERSION 4 (OPTIMIZADA):

```

#include "iter_subset.hpp"

/* Pre: Cert
Post: Construeix un iterador sobre els subconjunts de k elements
de {1, ..., n}; si k > n no hi ha res a recórrer. */
iter_subset::iter_subset(nat n, nat k) throw(error) { // cost  $\Theta(k)$ 
    _n = n;
    _k = k;
    _final = false;
    if (k > n) {
        _final = true;
        _actual.clear();
    } else {
        for (nat i = 0; i < k; ++i) {
            _actual.push_back(i + 1);
        }
    }
}

/* Tres grans. Constructor per còpia, operador d'assignació i
destructor. */

/* Pre: Cert
Post: Construeix una còpia exacta de l'iterador proporcionat. */
iter_subset::iter_subset(const iter_subset& its) throw(error) { // cost
 $\Theta(k)$ 
    _n = its._n;
    _k = its._k;
    _actual = its._actual;
    _final = its._final;
}

```

```

/* Pre: Cert
   Post: Assigna a l'iterador actual una còpia exacta de l'iterador
   proporcionat. */
iter_subset& iter_subset::operator=(const iter_subset& its)
throw(error) { // cost  $\Theta(k)$ 
    if (this != &its) {
        _n = its._n;
        _k = its._k;
        _actual = its._actual;
        _final = its._final;
    }
    return *this;
}

/* Pre: Cert
   Post: Destruïeix l'iterador alliberant qualsevol recurs associat. */
iter_subset::~~iter_subset() throw() {} // cost  $\Theta(1)$ 

/* Pre: Cert
   Post: Retorna cert si l'iterador ja ha visitat tots els subconjunts
   de k elements presos d'entre n; o dit d'una altra forma, retorna
   cert quan l'iterador apunta a un subconjunt sentinella fictici
   que queda a continuació de l'últim subconjunt vàlid. */
bool iter_subset::end() const throw() { // cost  $\Theta(1)$ 
    return _final;
}

/* Operador de desreferència.
   Pre: Cert
   Post: Retorna el subconjunt apuntat per l'iterador;
   llança un error si l'iterador apunta al sentinella. */
subset iter_subset::operator*() const throw(error) { // cost  $\Theta(1)$ 
    if (_final) {
        throw error(IterSubsetIncorr);
    }
    return _actual;
}

/* Operador de preincrement.
   Pre: Cert
   Post: Avança l'iterador al següent subconjunt en la seqüència i el
   retorna;

```



```

    no es produeix l'avançament si l'iterador ja apuntava al sentinella.
    */
iter_subset& iter_subset::operator++() throw() { // cost  $\Theta(k)$ 
    if (_final) {
        return *this;
    }

    subset ultimSubconjunt;
    for (nat i = _n - _k; i < _n; ++i) {
        ultimSubconjunt.push_back(i + 1);
    }

    if (_actual == ultimSubconjunt) {
        _final = true;
    } else {
        for (int i = _k - 1; i >= 0; --i) {
            if (_actual[i] < _n - (_k - 1 - i)) {
                ++_actual[i];
                for (nat j = i + 1; j < _k; ++j) {
                    _actual[j] = _actual[j - 1] + 1;
                }
                return *this;
            }
        }
        _final = true;
    }

    return *this;
}

/*Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i
retorna el seu valor
previ; no es produeix l'avançament si l'iterador ja apuntava al
sentinella. */
iter_subset iter_subset::operator++(int) throw() { // cost  $\Theta(k)$ 
    iter_subset temp(*this);
    if (!_final) {
        ++(*this);
    }
    return temp;
}

```

```

/* Pre: Cert
   Post: Retorna cert si dos iteradors són iguals (apunten al mateix
estat). */
bool iter_subset::operator==(const iter_subset& c) const throw() { //
cost Θ(k)
    if (_n != c._n and _k == 0 and c._k == 0 and _actual == c._actual and
_final == c._final) {
        return true;
    }
    return _n == c._n && _k == c._k && _actual == c._actual && _final ==
c._final;
}

/* Pre: Cert
   Post: Retorna cert si dos iteradors no són iguals. */
bool iter_subset::operator!=(const iter_subset& c) const throw() { //
cost Θ(k)
    return !(*this == c);
}

```

CLASE DICCIONARI

VERSION 1 (CON TRIE):

CPP:

```
#include "diccionari.hpp"
#include <string>
#include <list>
#include <vector>

// Constructor por defecto
diccionari::diccionari() throw(error) : _arrel(nullptr),
_num_paraules(0) {}

// Constructor por copia
diccionari::diccionari(const diccionari& D) throw(error)
    : _arrel(copia_tst(D._arrel)), _num_paraules(D._num_paraules) {}

// Operador de asignación
diccionari& diccionari::operator=(const diccionari& D) throw(error) {
    if (this != &D) {
        esborra_tst(_arrel);
        _arrel = copia_tst(D._arrel);
        _num_paraules = D._num_paraules;
    }
    return *this;
}

// Destructor
diccionari::~diccionari() throw() {
    esborra_tst(_arrel);
}

// Inserta una palabra en el diccionario
void diccionari::insereix(const std::string& p) throw(error) {
    if (p.empty()) return;
    bool afegida = false;
    _arrel = afegir_paraula(_arrel, p, 0, afegida);
    if (afegida) ++_num_paraules;
}

// Método auxiliar recursivo para insertar una palabra en el TST
```

```

diccionari::node* diccionari::afegir_paraula(node* n, const
std::string& paraula, int index, bool& afegida) {
    char c = paraula[index];
    if (!n) n = new node(c);

    if (c < n->_c) {
        n->_esq = afegir_paraula(n->_esq, paraula, index, afegida);
    } else if (c > n->_c) {
        n->_dre = afegir_paraula(n->_dre, paraula, index, afegida);
    } else {
        if (index + 1 == static_cast<int>(paraula.size())) {
            if (!n->_es_palabra) {
                n->_es_palabra = true;
                afegida = true;
            }
        } else {
            n->_cen = afegir_paraula(n->_cen, paraula, index + 1,
afegida);
        }
    }
    return n;
}

// Retorna el prefijo más largo de una palabra en el diccionario
std::string diccionari::prefix(const std::string& p) const throw(error)
{
    node* n = _arrel;
    std::string resultado, mejor_prefijo;

    int index = 0;
    while (n && index < static_cast<int>(p.size())) {
        char c = p[index];
        if (c < n->_c) {
            n = n->_esq;
        } else if (c > n->_c) {
            n = n->_dre;
        } else {
            resultado += c;
            if (n->_es_palabra) mejor_prefijo = resultado;
            n = n->_cen;
            ++index;
        }
    }
}

```

```

    return mejor_prefijo;
}

// Lista todas las palabras con longitud mayor o igual a k
void diccionari::llista_paraules(nat k, std::list<std::string>& L)
const throw(error) {
    std::string prefix_actual;
    llistar_paraules(_arrel, prefix_actual, k, L);
}

// Método auxiliar recursivo para listar palabras
void diccionari::llistar_paraules(node* n, std::string& prefix_actual,
nat k, std::list<std::string>& resultat) const {
    if (!n) return;

    llistar_paraules(n->_esq, prefix_actual, k, resultat);

    prefix_actual.push_back(n->_c);
    if (n->_es_palabra && prefix_actual.size() >= k) {
        resultat.push_back(prefix_actual);
    }
    llistar_paraules(n->_cen, prefix_actual, k, resultat);
    prefix_actual.pop_back();

    llistar_paraules(n->_dre, prefix_actual, k, resultat);
}

// Libera recursivamente el TST
void diccionari::esborra_tst(node* n) {
    if (!n) return;
    esborra_tst(n->_esq);
    esborra_tst(n->_cen);
    esborra_tst(n->_dre);
    delete n;
}

// Copia recursivamente el TST
diccionari::node* diccionari::copia_tst(node* n) {
    if (!n) return nullptr;
    node* nuevo = new node(n->_c);
    nuevo->_es_palabra = n->_es_palabra;
    nuevo->_esq = copia_tst(n->_esq);
    nuevo->_cen = copia_tst(n->_cen);

```

```

nuevo->_dre = copia_tst(n->_dre);
return nuevo;
}

// Retorna el número de palabras en el diccionario
nat diccionari::num_pal() const throw() {
    return _num_paraules;
}

```

REP:

```

struct TrieNode {
    TrieNode* fills[26]; // Array de punteros a los hijos (una
    posición por cada letra 'A'-'Z')
    bool es_palabra; // Indica si este nodo marca el final de una
    palabra

    // Constructor por defecto
    TrieNode() : es_palabra(false) {
        for (int i = 0; i < 26; ++i) {
            fills[i] = nullptr; // Inicializa todos los punteros como
            nullptr
        }
    }
};

// Nodo raíz del Trie
TrieNode* arrel;

// Número total de palabras almacenadas en el diccionario
nat num_paraules;

// Métodos auxiliares privados
void afegir_paraula(TrieNode* node, const std::string& paraula, int
index);
void satisfan_patro_rec(TrieNode* node, const std::vector<std::string>&
q, int index, std::string& palabra_actual, std::list<std::string>& L)
const;
void llistar_paraules(TrieNode* node, const std::string& prefix_actual,
nat k, std::list<std::string>& L) const;
void esborra_trie(TrieNode* node);
void afegir_copia(TrieNode* dest, TrieNode* src);

```

VERSION 2 (CON TRIE TST, COMO EN CLASE):

CPP:

```
#include "diccionari.hpp"
#include <string>
#include <list>
#include <vector>

// Constructor por defecto
diccionari::diccionari() throw(error) : _arrel(nullptr),
_num_paraules(0) {}

// Constructor por copia
diccionari::diccionari(const diccionari& D) throw(error)
    : _arrel(copia_tst(D._arrel)), _num_paraules(D._num_paraules) {}

// Operador de asignación
diccionari& diccionari::operator=(const diccionari& D) throw(error) {
    if (this != &D) {
        esborra_tst(_arrel);
        _arrel = copia_tst(D._arrel);
        _num_paraules = D._num_paraules;
    }
    return *this;
}

// Destructor
diccionari::~diccionari() throw() {
    esborra_tst(_arrel);
}

// Inserta una palabra en el diccionario
void diccionari::insereix(const string& p) throw(error) {
    if (p.empty()) return;
    bool afegida = false;
    _arrel = afegir_paraula(_arrel, p, 0, afegida);
    if (afegida) ++_num_paraules;
}

// Método auxiliar recursivo para insertar una palabra en el TST
```

```

diccionari::node* diccionari::afegir_paraula(node* n, const string&
paraula, int index, bool& afegida) {
    char c = paraula[index];
    if (!n) n = new node(c);

    if (c < n->_c) {
        n->_esq = afegir_paraula(n->_esq, paraula, index, afegida);
    } else if (c > n->_c) {
        n->_dre = afegir_paraula(n->_dre, paraula, index, afegida);
    } else {
        if (index + 1 == static_cast<int>(paraula.size())) {
            if (!n->_es_palabra) {
                n->_es_palabra = true;
                afegida = true;
            }
        } else {
            n->_cen = afegir_paraula(n->_cen, paraula, index + 1,
afegida);
        }
    }
    return n;
}

// Retorna el prefijo más largo de una palabra en el diccionario
string diccionari::prefix(const std::string& p) const throw(error) {
    node* n = _arrel;
    std::string resultado, mejor_prefijo;

    int index = 0;
    while (n && index < static_cast<int>(p.size())) {
        char c = p[index];
        if (c < n->_c) {
            n = n->_esq;
        } else if (c > n->_c) {
            n = n->_dre;
        } else {
            resultado += c;
            if (n->_es_palabra) mejor_prefijo = resultado;
            n = n->_cen;
            ++index;
        }
    }
    return mejor_prefijo;
}

```



```

}

// Lista todas las palabras con longitud mayor o igual a k
void diccionari::llista_paraules(nat k, list<string>& L) const
throw(error) {
    string prefix_actual;
    llistar_paraules(_arrel, prefix_actual, k, L);
}

// Método auxiliar recursivo para listar palabras
void diccionari::llistar_paraules(node* n, string& prefix_actual, nat
k, list<string>& resultat) const {
    if (!n) return;

    llistar_paraules(n->_esq, prefix_actual, k, resultat);

    prefix_actual.push_back(n->_c);
    if (n->_es_palabra && prefix_actual.size() >= k) {
        resultat.push_back(prefix_actual);
    }
    llistar_paraules(n->_cen, prefix_actual, k, resultat);
    prefix_actual.pop_back();

    llistar_paraules(n->_dre, prefix_actual, k, resultat);
}

// Libera recursivamente el TST
void diccionari::esborra_tst(node* n) {
    if (!n) return;
    esborra_tst(n->_esq);
    esborra_tst(n->_cen);
    esborra_tst(n->_dre);
    delete n;
}

// Copia recursivamente el TST
diccionari::node* diccionari::copia_tst(node* n) {
    if (!n) return nullptr;
    node* nuevo = new node(n->_c);
    nuevo->_es_palabra = n->_es_palabra;
    nuevo->_esq = copia_tst(n->_esq);
    nuevo->_cen = copia_tst(n->_cen);
    nuevo->_dre = copia_tst(n->_dre);
}

```

```

        return nuevo;
    }

    // Encuentra todas las palabras que cumplen con el patrón especificado
    void diccionari::satisfan_patro(const vector<string>& q, list<string>&
L) const throw(error) {
        string palabra_actual;
        satisfan_patro_rec(_arrel, q, 0, palabra_actual, L);
    }

    // Método auxiliar recursivo para buscar palabras que cumplen el patrón
    void diccionari::satisfan_patro_rec(node* n, const vector<string>& q,
int index, string& palabra_actual, list<string>& L) const {
        if (!n || index >= static_cast<int>(q.size())) return;

        // Procesa el hijo izquierdo (caracteres menores)
        satisfan_patro_rec(n->_esq, q, index, palabra_actual, L);

        // Comprueba si el carácter actual coincide con el patrón
        if (q[index].find(n->_c) != std::string::npos) {
            palabra_actual.push_back(n->_c);
            if (index == static_cast<int>(q.size()) - 1 && n->_es_palabra) {
                L.push_back(palabra_actual);
            } else {
                satisfan_patro_rec(n->_cen, q, index + 1, palabra_actual,
L);
            }
            palabra_actual.pop_back();
        }

        // Procesa el hijo derecho (caracteres mayores)
        satisfan_patro_rec(n->_dre, q, index, palabra_actual, L);
    }

    // Devuelve el número de palabras en el diccionario
    nat diccionari::num_pal() const throw() {
        return _num_paraules;
    }

```

REP:

```

struct node {
    char _c;           // Carácter en la posición actual de la palabra

```

```

        node* _esq;          // Hijo izquierdo: caracteres menores en la
misma posición
        node* _cen;          // Hijo central: siguiente carácter en la
palabra
        node* _dre;          // Hijo derecho: caracteres mayores en la misma
posición
        bool _es_palabra;    // Indica si termina una palabra en este nodo

        // Constructor
        node(char c) : _c(c), _esq(nullptr), _cen(nullptr),
        _dre(nullptr), _es_palabra(false) {}
    };

node* _arrel;    // Raíz del Ternary Search Tree
nat _num_paraules; // Número total de palabras en el diccionario

// Métodos auxiliares privados
node* afegir_paraula(node* n, const string& paraula, int index, bool&
afegida);
void esborra_tst(node* n);
node* copia_tst(node* n);
void llistar_paraules(node* n, string& prefix_actual, nat k,
list<string>& resultat) const;
void satisfan_patro_rec(node* n, const vector<string>& q, int index,
string& palabra_actual, list<string>& L) const;

```

VERSION 3 (OPTIMIZADA):

```

#include "diccionari.hpp"

/* Pre: Cert
   Post: Construeix un diccionari que conté únicament una paraula:
   la paraula buida. */
diccionari::diccionari() throw(error) { // cost  $\Theta(1)$ 
    _arrel = nullptr;
    _num_paraules = 0;
}

/* Pre: Cert
   Post: Construeix un diccionari que és una còpia del diccionari D. */
diccionari::diccionari(const diccionari& D) throw(error) { // cost  $\Theta(n)$ 
    _arrel = copia_tst(D._arrel); // Còpia profunda de l'arbre TST
    _num_paraules = D._num_paraules;
}

```

```

}

/* Pre: Cert
   Post: Assigna al diccionari actual una còpia del diccionari D. */
diccionari& diccionari::operator=(const diccionari& D) throw(error) {
// cost  $\Theta(n + m)$ 
    if (this != &D) {
        esborra_tst(_arrel); // Esborra l'arbre TST actual
        _arrel = copia_tst(D._arrel); // Còpia profunda de l'arbre TST
        _num_paraules = D._num_paraules;
    }
    return *this;
}

/* Pre: Cert
   Post: Destruïx el diccionari alliberant tota la memòria ocupada. */
diccionari::~~diccionari() throw() { // cost  $\Theta(n)$ 
    esborra_tst(_arrel); // Allibera la memòria ocupada per l'arbre TST
}

/* Pre: Cert
   Post: Retorna una còpia de l'arbre TST amb arrel n. */
diccionari::node* diccionari::copia_tst(node* n) { // cost  $\Theta(n)$ 
    if (!n) {
        return nullptr;
    }

    node* nou = new node(n->_c);
    nou->_es_paraula = n->_es_paraula; // Còpia de l'estat de la paraula
    // Còpia dels subarbres
    nou->_esq = copia_tst(n->_esq);
    nou->_cen = copia_tst(n->_cen);
    nou->_dre = copia_tst(n->_dre);

    return nou;
}

/* Pre: Cert
   Post: Allibera tota la memòria ocupada per l'arbre TST amb arrel n.
   */
void diccionari::esborra_tst(node* n) { // cost  $\Theta(n)$ 
    if (!n) {
        return;
    }

```

```

    }

    // Alliberem els subarbres
    esborra_tst(n->_esq);
    esborra_tst(n->_cen);
    esborra_tst(n->_dre);

    delete n; // Allibera el node actual
}

/* Pre: Cert
   Post: Afegeix la paraula p al diccionari; si la paraula p ja
   formava part del diccionari, l'operació no té cap efecte. */
void diccionari::insereix(const string& p) throw(error) { // cost  $\Theta(k)$ 
    if (p.empty()) { // No afegir si la paraula és buida
        return;
    }

    bool afegida = false;
    _arrel = afegir_paraula(_arrel, p, 0, afegida);

    if (afegida) {
        ++_num_paraules; // Incrementa el compte si s'ha afegit
    }
}

/* Pre: Cert
   Post: Retorna l'arbre TST resultant d'afegir la paraula paraula
   al subarbre amb arrel n. Si s'afegeix una nova paraula, afegida
   passa a ser true. */
diccionari::node* diccionari::afegir_paraula(node* n, const string&
paraula, nat index, bool& afegida) { // cost  $\Theta(k)$ 
    if (index == paraula.size()) {
        return n; // Si s'ha processat tota la paraula
    }

    char c = paraula[index];
    if (!n) {
        n = new node(c); // Crea un nou node si no existeix
    }

    if (c < n->_c) {
        n->_esq = afegir_paraula(n->_esq, paraula, index, afegida);
    } else if (c > n->_c) {

```

```

    n->_dre = afegir_paraula(n->_dre, paraula, index, afegida);
} else {
    if (index + 1 == paraula.size()) {
        if (!n->_es_paraula) {
            n->_es_paraula = true; // Marca com a paraula completa
            afegida = true; // Indica que s'ha afegit una nova paraula
        }
    } else {
        n->_cen = afegir_paraula(n->_cen, paraula, index + 1, afegida);
    }
}
return n;
}

/* Pre: Cert
   Post: Retorna el prefix més llarg de p que és una paraula que
   pertany al diccionari. */
string diccionari::prefix(const string& p) const throw(error) { // cost
    Θ(k)
    node* n = _arrel;
    string millor_prefix;
    string actual;

    for (nat index = 0; n and index < p.size(); ) {
        char c = p[index];
        if (c < n->_c) {
            n = n->_esq;
        } else if (c > n->_c) {
            n = n->_dre;
        } else {
            actual.push_back(c);
            if (n->_es_paraula) {
                millor_prefix = actual; // Actualitza el millor prefix
            }
            n = n->_cen;
            ++index;
        }
    }
    return millor_prefix;
}

/* Pre: Cert
   Post: Retorna la llista de paraules del diccionari que satisfan

```

```

    el patró especificat en el vector d'strings q, en ordre
    alfabètic ascendent. */
void diccionari::satisfan_patro(const vector<string>& q, list<string>&
L) const throw(error) { // cost  $\Theta(n * m)$ 
    string palabra_actual;
    satisfan_patro_rec(_arrel, q, 0, palabra_actual, L);
}

/* Pre: Cert
   Post: Afegeix a L totes les paraules del subarbre amb arrel n que
   satisfan el patró q[index..q.size()-1]. */
void diccionari::satisfan_patro_rec(node* n, const vector<string>& q,
nat index, string& palabra_actual, list<string>& L) const { // cost  $\Theta(n
* m)$ 
    if (!n or index >= q.size()) {
        return;
    }

    satisfan_patro_rec(n->_esq, q, index, palabra_actual, L);

    for (nat i = 0; i < q[index].size(); ++i) {
        if (q[index][i] == n->_c) {
            palabra_actual.push_back(n->_c);
            if (index == q.size() - 1 and n->_es_paraula) {
                L.push_back(palabra_actual); // Afegeix la paraula a la
llista si compleix el patró
            } else {
                satisfan_patro_rec(n->_cen, q, index + 1, palabra_actual,
L);
            }
            palabra_actual.pop_back();
            break;
        }
    }

    satisfan_patro_rec(n->_dre, q, index, palabra_actual, L);
}

/* Pre: Cert
   Post: Retorna una llista amb totes les paraules del diccionari
   de longitud major o igual a k en ordre alfabètic ascendent. */
void diccionari::llista_paraules(nat k, list<string>& L) const
throw(error) { // cost  $\Theta(n)$ 

```

```

    string prefix_actual;
    llistar_paraules(_arrel, prefix_actual, k, L);
}

/* Pre: Cert
   Post: Afegeix a resultat totes les paraules del subarbre amb arrel n
         de longitud major o igual a k. */
void diccionari::llistar_paraules(node* n, string& prefix_actual, nat
k, list<string>& resultat) const { // cost  $\Theta(n)$ 
    if (!n) {
        return;
    }

    llistar_paraules(n->_esq, prefix_actual, k, resultat);
    prefix_actual.push_back(n->_c);
    if (n->_es_paraula and prefix_actual.size() >= k) {
        resultat.push_back(prefix_actual); // Afegeix la paraula si
compleix la longitud
    }
    llistar_paraules(n->_cen, prefix_actual, k, resultat);
    prefix_actual.pop_back();
    llistar_paraules(n->_dre, prefix_actual, k, resultat);
}

/* Pre: Cert
   Post: Retorna el nombre de paraules en el diccionari. */
nat diccionari::num_pal() const throw() { // cost  $\Theta(1)$ 
    return _num_paraules;
}

```


CLASE ANAGRAMES

VERSION 1 (Violació de segment):

```
#include "anagrames.hpp"
#include "word_toolkit.hpp"
#include <stdexcept>
#include <algorithm>
#include <climits> // Para LONG_MAX

/* Pre: Cert
Post: Construeix un anagrama buit. */
anagrames::anagrames() throw(error) : _M(101), _quants(0) {
    _taula.resize(_M, nullptr); // Inicializa la tabla con tamaño _M
}

/* Tres grans. Constructor per còpia, operador d'assignació i
destructor. */
anagrames::anagrames(const anagrames& A) throw(error) : _M(A._M),
    _quants(A._quants) {
    _taula.resize(_M, nullptr);
    for (size_t i = 0; i < _M; ++i) {
        if (A._taula[i]) {
            _taula[i] = copia_nodos_anagrames(A._taula[i]); // Copiar
nodos
        }
    }
}

anagrames& anagrames::operator=(const anagrames& A) throw(error) {
    if (this != &A) {
        for (size_t i = 0; i < _M; ++i) {
            esborra_nodos(_taula[i]); // Liberar nodos actuales
        }
    }
}
```

```

        _M = A._M;
        _quants = A._quants;
        _taula.resize(_M, nullptr);

        for (size_t i = 0; i < _M; ++i) {
            if (A._taula[i]) {
                _taula[i] = copia_nodes_anagrames(A._taula[i]);
            }
        }
    }
    return *this;
}

anagrames::~~anagrames() throw() {
    for (size_t i = 0; i < _M; ++i) {
        esborra_nodes(_taula[i]);
    }
}

/* Pre: Cert
   Post: Afegeix la paraula p a l'anagrama; si la paraula p ja
   formava part de l'anagrama, l'operació no té cap efecte. */
void anagrames::insereix(const string& p) throw(error) {
    string anagrama = word_toolkit::anagrama_canonic(p);
    size_t index = h(anagrama) % _M;

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == anagrama) {
            if (find(actual->_paraules.begin(), actual->_paraules.end(),
p) == actual->_paraules.end()) {
                actual->_paraules.push_back(p);
            }
            return;
        }
        actual = actual->_seg;
    }

    node_hash* nuevo = new node_hash(anagrama, {p}, _taula[index]);
    _taula[index] = nuevo;
    ++_quants;
}

```

```

/* Pre:  Cert
   Post: Retorna la llista de paraules p tals que anagrama_canonic(p)=a.
   Llança un error si les lletres de a no estan en ordre ascendent. */
void anagrames::mateix_anagrama_canonic(const string& a, list<string>&
L) const throw(error) {
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic);
    }

    size_t index = h(a) % _M;
    node_hash* actual = _taula[index];

    while (actual) {
        if (actual->_clau == a) {
            L = actual->_paraules;
            return;
        }
        actual = actual->_seg;
    }

    L.clear(); // Si no se encuentra, la lista estará vacía
}

/* Función hash para calcular el índice */
long anagrames::h(const string& clau) const {
    long valor = 0;
    for (char c : clau) {
        valor = (MULT * valor + c) % LONG_MAX;
    }
    return valor;
}

/* Libera todos los nodos en una cubeta */
void anagrames::esborra_nodos(node_hash* p) {
    while (p) {
        node_hash* temp = p;
        p = p->_seg;
        delete temp;
    }
}

/* Realiza una copia profunda de una lista de nodos */

```

```

anagrames::node_hash* anagrames::copia_nodes_anagrames(const node_hash*
p) {
    if (!p) return nullptr;

    node_hash* nuevo = new node_hash(p->_clau, p->_paraules, nullptr);
    node_hash* actual = nuevo;

    while (p->_seg) {
        p = p->_seg;
        actual->_seg = new node_hash(p->_clau, p->_paraules, nullptr);
        actual = actual->_seg;
    }

    return nuevo;
}

```

VERSION 2 (Excepción de coma flotante):

```

#include "anagrames.hpp"
#include "word_toolkit.hpp"
#include <stdexcept>
#include <algorithm>
#include <climits> // Para LONG_MAX

/* Pre: Cert
Post: Construeix un anagrama buit. */
anagrames::anagrames() throw(error) : _M(101), _quants(0) {
    _taula.resize(_M, nullptr); // Inicializa la tabla con tamaño _M
}

/* Tres grans. Constructor per còpia, operador d'assignació i
destructor. */
anagrames::anagrames(const anagrames& A) throw(error) : _M(A._M),
_quants(A._quants) {
    _taula.resize(_M, nullptr);
    for (size_t i = 0; i < _M; ++i) {
        if (A._taula[i]) {
            _taula[i] = copia_nodes_anagrames(A._taula[i]); // Copiar
nodos
        }
    }
}

```

```

anagrames& anagrames::operator=(const anagrames& A) throw(error) {
    if (this != &A) {
        for (size_t i = 0; i < _M; ++i) {
            esborra_nodos(_taula[i]); // Liberar nodos actuaes
        }

        _M = A._M;
        _quants = A._quants;
        _taula.resize(_M, nullptr);

        for (size_t i = 0; i < _M; ++i) {
            if (A._taula[i]) {
                _taula[i] = copia_nodos_anagrames(A._taula[i]);
            }
        }
    }
    return *this;
}

anagrames::~anagrames() throw() {
    for (size_t i = 0; i < _M; ++i) {
        esborra_nodos(_taula[i]);
    }
}

/* Pre: Cert
   Post: Afegeix la paraula p a l'anagrama; si la paraula p ja
   formava part de l'anagrama, l'operació no té cap efecte. */
void anagrames::insereix(const string& p) throw(error) {
    string anagrama = word_toolkit::anagrama_canonic(p);
    size_t index = h(anagrama) % _M;

    if (_taula[index] == nullptr) {
        _taula[index] = new node_hash(anagrama, {p}, nullptr);
        ++_quants;
        return;
    }

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == anagrama) {
            if (find(actual->_paraules.begin(), actual->_paraules.end(),
p) == actual->_paraules.end()) {

```

```

        actual->_paraules.push_back(p);
    }
    return;
}
actual = actual->_seg;
}

// Insertar nuevo nodo al inicio de la lista enlazada
node_hash* nuevo = new node_hash(anagrama, {p}, _taula[index]);
_taula[index] = nuevo;
++_quants;
}

/* Pre: Cert
Post: Retorna la llista de paraules p tals que anagrama_canonic(p)=a.
Llança un error si les lletres de a no estan en ordre ascendent. */
void anagrames::mateix_anagrama_canonic(const string& a, list<string>&
L) const throw(error) {
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic);
    }

    size_t index = h(a) % _M;

    if (_taula[index] == nullptr) {
        L.clear(); // Si no hay nodos en esta cubeta, devolvemos lista
vacía
        return;
    }

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == a) {
            L = actual->_paraules;
            return;
        }
        actual = actual->_seg;
    }

    L.clear(); // Si no se encuentra, la lista estará vacía
}

/* Función hash para calcular el índice */

```

```

long anagrames::h(const string& clau) const {
    long valor = 0;
    for (char c : clau) {
        valor = (MULT * valor + c) % LONG_MAX;
    }
    return (valor >= 0 ? valor : -valor); // Garantiza que el índice sea
no negativo
}

/* Libera todos los nodos en una cubeta */
void anagrames::esborra_nodos(node_hash* p) {
    while (p) {
        node_hash* temp = p;
        p = p->_seg;
        delete temp;
    }
}

/* Realiza una copia profunda de una lista de nodos */
anagrames::node_hash* anagrames::copia_nodos_anagrames(const node_hash*
p) {
    if (!p) return nullptr;

    node_hash* nuevo = new node_hash(p->_clau, p->_paraules, nullptr);
    node_hash* actual = nuevo;

    while (p->_seg) {
        p = p->_seg;
        actual->_seg = new node_hash(p->_clau, p->_paraules, nullptr);
        actual = actual->_seg;
    }

    return nuevo;
}

```

VERSION 3 (free(): invalid size):

```

#include "anagrames.hpp"
#include "word_toolkit.hpp"
#include <stdexcept>
#include <algorithm>

```

```

#include <climits> // Para LONG_MAX

anagrames::anagrames() throw(error) : _M(101), _quants(0) {
    if (_M == 0) {
        throw error(NoEsCanonic);
    }
    _taula.resize(_M, nullptr);
}

anagrames::anagrames(const anagrames& A) throw(error) : _M(A._M),
_quants(A._quants) {
    _taula.resize(_M, nullptr);
    for (size_t i = 0; i < _M; ++i) {
        if (A._taula[i]) {
            _taula[i] = copia_nodes_anagrames(A._taula[i]);
        }
    }
}

anagrames& anagrames::operator=(const anagrames& A) throw(error) {
    if (this != &A) {
        for (size_t i = 0; i < _M; ++i) {
            esborra_nodes(_taula[i]);
        }

        _M = A._M;
        _quants = A._quants;
        _taula.resize(_M, nullptr);

        for (size_t i = 0; i < _M; ++i) {
            if (A._taula[i]) {
                _taula[i] = copia_nodes_anagrames(A._taula[i]);
            }
        }
    }
    return *this;
}

anagrames::~~anagrames() throw() {
    for (size_t i = 0; i < _M; ++i) {
        esborra_nodes(_taula[i]);
    }
}

```



```

void anagrames::insereix(const string& p) throw(error) {
    if (p.empty()) {
        throw error(NoEsCanonic);
    }

    string anagrama = word_toolkit::anagrama_canonic(p);
    size_t index = (_M > 0) ? h(anagrama) % _M : 0;

    if (_taula[index] == nullptr) {
        _taula[index] = new node_hash(anagrama, {p}, nullptr);
        ++_quants;
        return;
    }

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == anagrama) {
            if (find(actual->_paraules.begin(), actual->_paraules.end(),
p) == actual->_paraules.end()) {
                actual->_paraules.push_back(p);
            }
            return;
        }
        actual = actual->_seg;
    }

    node_hash* nuevo = new node_hash(anagrama, {p}, _taula[index]);
    _taula[index] = nuevo;
    ++_quants;
}

void anagrames::mateix_anagrama_canonic(const string& a, list<string>&
L) const throw(error) {
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic);
    }

    size_t index = (_M > 0) ? h(a) % _M : 0;

    if (_taula[index] == nullptr) {
        L.clear();
        return;
    }

```

```

    }

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == a) {
            L = actual->_paraules;
            return;
        }
        actual = actual->_seg;
    }

    L.clear();
}

long anagrames::h(const string& clau) const {
    if (clau.empty()) {
        throw error(NoEsCanonic);
    }

    long valor = 0;
    for (char c : clau) {
        valor = (MULT * valor + c) % LONG_MAX;
    }
    return (valor >= 0 ? valor : -valor);
}

void anagrames::esborra_nodes(node_hash* p) {
    while (p) {
        node_hash* temp = p;
        p = p->_seg;
        delete temp;
    }
}

anagrames::node_hash* anagrames::copia_nodes_anagrames(const node_hash*
p) {
    if (!p) return nullptr;

    node_hash* nuevo = new node_hash(p->_clau, p->_paraules, nullptr);
    node_hash* actual = nuevo;

    while (p->_seg) {
        p = p->_seg;
    }
}

```

```

        actual->_seg = new node_hash(p->_clau, p->_paraules, nullptr);
        actual = actual->_seg;
    }

    return nuevo;
}

```

VERSION 4 (FUNCIONA TODO PERO CON MAP):

CPP:

```

#include "anagrames.hpp"
#include "word_toolkit.hpp"
#include <stdexcept>
#include <algorithm>

/* Pre: Cert
   Post: Construeix un anagrama buit. */
anagrames::anagrames() throw(error) {
    _rep = new rep(); // Crea la representació
}

/* Constructor per còpia */
anagrames::anagrames(const anagrames& A) throw(error) {
    _rep = new rep();
    _rep->taula = A._rep->taula; // Copia la taula d'anagrames
    _rep->quants = A._rep->quants;
}

/* Operador d'assignació */
anagrames& anagrames::operator=(const anagrames& A) throw(error) {
    if (this != &A) {
        *_rep = *A._rep;
    }
    return *this;
}

/* Destructor */
anagrames::~~anagrames() throw() {
    delete _rep; // Allibera la representació
}

/* Pre: Cert

```

```

    Post: Afegeix la paraula p a l'anagrama i al diccionari. */
void anagrames::insereix(const string& p) throw(error) {
    // 1. Inserir al diccionari base
    diccionari::insereix(p);

    // 2. Obtenir l'anagrama canònic
    string anagrama_c = word_toolkit::anagrama_canonic(p);

    // 3. Inserir la paraula a la taula d'anagrames si no hi és
    list<string>& paraules = _rep->taula[anagrama_c];
    if (find(paraules.begin(), paraules.end(), p) == paraules.end()) {
        paraules.push_back(p);
        ++_rep->quants; // Incrementa el comptador
    }
}

/* Pre: Cert
   Post: Retorna la llista de paraules amb el mateix anagrama canònic
   'a'. */
void anagrames::mateix_anagrama_canonic(const string& a, list<string>&
L) const throw(error) {
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic); // Llança un error si 'a' no és
canònic
    }

    // Cerca a la taula d'anagrames
    auto it = _rep->taula.find(a);
    if (it != _rep->taula.end()) {
        L = it->second; // Retorna la llista associada
        L.sort();      // Ordena lexicogràficament
    } else {
        L.clear();     // No s'ha trobat cap paraula
    }
}

```

REP:

```

class rep { // Representación interna de anagrames
public:
    map<string, list<string>> taula; // Clave: anagrama canónico, Valor: lista de
palabras
    size_t quants;                  // Número total de palabras insertadas

```

diccionari dicc;

rep() : quants(0) {} // Constructor por defecto
};

rep* _rep;

VERSION 5 (OPTIMIZADA):

CPP:

```
#include "anagrames.hpp"
#include "word_toolkit.hpp"

/* Pre: Cert
   Post: Intercanvia els valors de 'a' i 'b'. */
void swap(char& a, char& b) { // cost  $\Theta(1)$ 
    char temporal = a;
    a = b;
    b = temporal;
}

/* Pre: Cert
   Post: Particiona la cadena 's' al voltant del pivot i retorna la seva
   posició. */
int particio(string& s, int esquerra, int dreta) { // cost  $\Theta(n)$ 
    char pivot = s[dreta];
    int i = esquerra - 1;

    for (int j = esquerra; j < dreta; ++j) {
        if (s[j] <= pivot) {
            ++i;
            swap(s[i], s[j]);
        }
    }
    swap(s[i + 1], s[dreta]); // Col·loca el pivot en la seva posició
    correcta
    return i + 1;
}

/* Pre: esquerra <= dreta i 's' és una cadena vàlida.
   Post: Ordena la cadena 's' en ordre ascendent entre les posicions
   'esquerra' i 'dreta'. */
```

```

void quicksort(string& s, int esquerra, int dreta) { // cost  $\Theta(n \cdot \log n)$  promedi, pitjor cas es cost  $\Theta(n^2)$ 
    if (esquerra < dreta) {
        int pivot = particio(s, esquerra, dreta);
        quicksort(s, esquerra, pivot - 1); // Ordena la part esquerra del pivot
        quicksort(s, pivot + 1, dreta); // Ordena la part dreta del pivot
    }
}

/* Pre: Cert
   Post: Construeix un anagrama buit. */
anagrames::anagrames() throw(error) { // cost  $\Theta(1)$ 
    taula = new node*[101]();
    mida = 0;
    quants = 101;
}

/* Pre: Cert
   Post: Construeix un nou objecte idèntic a 'other'. */
anagrames::anagrames(const anagrames& other) throw(error) { // cost  $\Theta(n)$ 
    taula = new node*[other.quants]();
    mida = other.mida;
    quants = other.quants;
    for (nat i = 0; i < quants; ++i) {
        if (other.taula[i]) {
            node* actual = other.taula[i];
            node** this_actual = &taula[i];

            while (actual) {
                *this_actual = new node(actual->clau); // Crea un nou node amb la mateixa clau
                (*this_actual)->paraules = actual->paraules; // Copia la llista de paraules
                actual = actual->seg;
                this_actual = &((*this_actual)->seg); // Avança al següent node
            }
        }
    }
}

```

```

/* Pre: Cert
   Post: Assigna a aquest objecte els valors d''other'. */
anagrames& anagrames::operator=(const anagrames& other) throw(error) {
// cost  $\Theta(n)$ 
    if (this != &other) {
        for (nat i = 0; i < quants; ++i) {
            node* n = taula[i];
            while (n) {
                node* temp = n;
                n = n->seg;
                delete temp; // Allibera la memòria dels nodes actuals
            }
        }
        delete[] taula;

        taula = new node*[other.quants]();
        mida = other.mida;
        quants = other.quants;

        for (nat i = 0; i < quants; ++i) {
            if (other.taula[i]) {
                node* actual = other.taula[i];
                node** this_actual = &taula[i];

                while (actual) {
                    *this_actual = new node(actual->clau); // Crea un
nou node amb la mateixa clau
                    (*this_actual)->paraules = actual->paraules; //
Copia la llista de paraules
                    actual = actual->seg;
                    this_actual = &((*this_actual)->seg); // Avança al
següent node
                }
            }
        }
    }
    return *this;
}

/* Pre: Cert
   Post: Allibera tota la memòria dinàmica associada a l'objecte. */
anagrames::~~anagrames() { // cost  $\Theta(n)$ 

```

```

    for (nat i = 0; i < quants; ++i) {
        node* n = taula[i];
        while (n != nullptr) {
            node* temp = n;
            n = n->seg;
            delete temp; // Allibera la memòria de cada node
        }
        taula[i] = nullptr;
    }
    delete[] taula;
    taula = nullptr; // Allibera la memòria de la taula
}

/* Pre: Cert
   Post: Afegeix la paraula p al diccionari; si la paraula p ja
   formava part del diccionari, l'operació no té cap efecte. */
void anagrames::insereix(const string& p) throw(error) { // cost  $\Theta(p + m)$ 

    diccionari::insereix(p); // Insereix la paraula al diccionari

    string p_ordenada = p;
    quicksort(p_ordenada, 0, p_ordenada.size() - 1); // Ordena la
paraula

    nat index = h(p_ordenada);
    node* n = taula[index];

    while (n != nullptr) {
        if (n->clau == p_ordenada) {
            for (list<string>::iterator it = n->paraules.begin(); it !=
n->paraules.end(); ++it) {
                if (*it == p) return; // No afegeix si la paraula ja
existeix
            }
            n->paraules.push_back(p); // Afegeix la paraula a la llista
            return;
        }
        n = n->seg;
    }

    node* nou_node = new node(p_ordenada);
    nou_node->paraules.push_back(p); // Afegeix la paraula al nou node
    nou_node->seg = taula[index];

```



```

    taula[index] = nou_node;
    ++mida;

    if (mida > quants * 0.75) { // Redistribueix si la taula està plena
        redispersio();
    }
}

/* Pre: Cert
   Post: Retorna la llista de paraules p tals que anagrama_canonic(p)=a
   ordenades en ordre ascendent.
   Llança un error si les lletres de a no estan en ordre ascendent. */
void anagrames::mateix_anagrama_canonic(const string& a, list<string>&
L) const throw(error) { // cost  $\Theta(L * p + m * L + m * \log L)$ 
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic);
    }

    for (list<string>::iterator it = L.begin(); it != L.end(); ) {
        string it_ordenada = *it;
        quicksort(it_ordenada, 0, it_ordenada.size() - 1);
        if (it_ordenada != a) {
            it = L.erase(it); // Elimina les paraules que no són
anagrames canònics
        } else {
            ++it;
        }
    }

    nat index = h(a);
    node* n = taula[index];

    while (n != nullptr) {
        if (n->clau == a) {
            for (list<string>::iterator it_paraula =
n->paraules.begin(); it_paraula != n->paraules.end(); ++it_paraula) {
                bool trobat = false;
                for (list<string>::iterator it_existent = L.begin();
it_existent != L.end(); ++it_existent) {
                    if (*it_existent == *it_paraula) {
                        trobat = true;
                        break; // Surt del bucle si es troba la paraula
                    }
                }
            }
        }
    }
}

```

```

        }
        if (!trobat) {
            L.push_back(*it_paraula); // Afegeix les paraules
que no existeixen
        }
    }
    L.sort(); // Ordena la llista de paraules
    return;
}
n = n->seg;
}
L.clear(); // Neteja la llista si no es troben anagrames
}

/* Pre: Cert
   Post: Retorna la posició de hash per a la clau donada. */
nat anagrames::h(const string& clau) const { // cost  $\Theta(p)$ 
    nat hash = 0;
    for (nat i = 0; i < clau.size(); ++i) {
        hash = (hash * 31 + clau[i]) % quants; // Calcula l'índex de
hash
    }
    return hash;
}

/* Pre: Cert
   Post: Redistribueix els elements de la taula hash en una nova taula
de mida doble. */
void anagrames::redispersio() { // cost  $\Theta(n + m * p)$ 
    nat nou_quants = quants * 2;
    node** nova_taula = new node*[nou_quants]();

    for (nat i = 0; i < quants; ++i) {
        node* n = taula[i];
        while (n != nullptr) {
            node* temp = n;
            n = n->seg;

            nat nou_index = 0;
            for (size_t j = 0; j < temp->clau.size(); ++j) {
                nou_index = (nou_index * 31 + temp->clau[j]) %
nou_quants; // Calcula el nou índex de hash
            }

```

```
        temp->seg = nova_taula[nou_index];  
        nova_taula[nou_index] = temp; // Reubica el node en la nova  
taula  
    }  
}  
  
delete[] taula;  
taula = nova_taula;  
quants = nou_quants; // Actualitza la mida de la taula  
}
```

CLASE OBTÉ PARAULES

VERSION 1 (NO SALEN ORDENADAS):

```

#include "obte_paraules.hpp"
#include "iter_subset.hpp" // Incluye la declaración de iter_subset y
subset
#include "word_toolkit.hpp" // Incluye el namespace word_toolkit

/* Pre: Cert
   Post: Retorna la llista de paraules que es poden formar usant k
lletres
   de la paraula s. Llança error si k és major que la longitud de
   l'string s o k < 3. */
void obte_paraules::obte_paraules(util::nat k, const std::string& s,
const anagrames& A, std::list<std::string>& paraules) throw(error) {
    if (k > s.size() || k < 3) {
        throw error(LongitudInvalida); // Validació de precondicions
    }

    paraules = std::list<std::string>(); // Inicialitzem la llista
directament

    iter_subset subconjunts(s.size(), k); // Creem iterador per als
subconjunts de k elements
    while (!subconjunts.end()) { // Mentre quedin subconjunts
        const subset& indexos = *subconjunts; // Obtenim el subconjunt
actual

        // Construïm una paraula utilitzant els índexs del subconjunt
        std::string paraula_actual;
        for (util::nat idx : indexos) {
            paraula_actual += s[idx - 1]; // Convertim l'índex (1-based)
al caràcter corresponent
        }

        // Obtenim l'anagrama canònic de la paraula construïda
        std::string canonic =
word_toolkit::anagrama_canonic(paraula_actual);

        // Obtenim les paraules associades a l'anagrama canònic
        std::list<std::string> paraules_anagrames;
        A.mateix_anagrama_canonic(canonic, paraules_anagrames);

        // Afegim les paraules a la llista principal sense fer servir
`insert`

```

```

        for (const std::string& p : paraules_anagrames) {
            paraules.push_back(p);
        }

        ++subconjunts; // Avancem al següent subconjunt
    }
}

/* Pre: Cert
   Post: Retorna la llista de paraules que es poden formar usant 3 o més
   lletres
   de la paraula s. La llista estarà ordenada de menys a més longitud;
   a igual longitud les paraules estan en ordre alfabètic creixent.
   Llança un error si l'string s té menys de tres lletres. */
void obte_paraules::obte_paraules(const std::string& s, const
anagrames& A, std::list<std::string>& paraules) throw(error) {
    if (s.size() < 3) {
        throw error(LongitudInvalida); // Validació de precondicions
    }

    paraules = std::list<std::string>(); // Inicialitzem la llista
directament

    // Iterem per totes les longituds possibles (de 3 fins a la longitud
de la paraula)
    for (util::nat k = 3; k <= s.size(); ++k) {
        std::list<std::string> resultats_parcials;

        // Fem servir la funció anterior per obtenir paraules de
longitud k
        obte_paraules(k, s, A, resultats_parcials);

        // Afegim les paraules de resultats_parcials a la llista
principal
        for (const std::string& p : resultats_parcials) {
            paraules.push_back(p);
        }
    }
}

```

VERSION 2 :

```
#include "obte_paraules.hpp"
```

```

#include "iter_subset.hpp"
#include "word_toolkit.hpp"

/* Pre: Cert
   Post: Retorna la llista de paraules que es poden formar usant k
   lletres
   de la paraula s. Llança error si k és major que la longitud de
   l'string s o k < 3. */
void obte_paraules::obte_paraules(nat k, const string& s, const
anagrames& A, list<string>& paraules) throw(error) {
    // Inicializamos `paraules` como una lista vacía
    paraules = list<std::string>();

    // Validación de precondiciones
    if (k > s.size() or k < 3) {
        throw error(LongitudInvalida);
    }

    iter_subset subconjunts(s.size(), k); // Crear iterador para
subconjuntos de k elementos
    while (!subconjunts.end()) { // Mientras queden subconjuntos
        const subset& indexs = *subconjunts; // Obtener el subconjunto
actual

        // Construir una palabra utilizando los índices del subconjunto
        string paraula_actual;
        for (nat i = 0; i < indexs.size(); ++i) {
            paraula_actual += s[indexs[i] - 1]; // Convertir índice
(1-based) a carácter
        }

        // Obtener el anagrama canónico de la palabra construida
        string canonic = word_toolkit::anagrama_canonic(paraula_actual);

        // Obtener las palabras asociadas al anagrama canónico
        list<string> paraules_anagrames;
        A.mateix_anagrama_canonic(canonic, paraules_anagrames);

        // Añadir palabras a la lista principal manualmente, evitando
duplicados
        while (!paraules_anagrames.empty()) {
            string p = paraules_anagrames.front(); // Extraer la primera
palabra

```

```

        paraules_anagrames.pop_front(); // Eliminar la palabra de la
lista original

        // Comprobar si la palabra ya existe en la lista
        bool ja_existeix = false;
        for (list<string>::iterator it = paraules.begin(); it !=
paraules.end(); ++it) {
            if (*it == p) {
                ja_existeix = true; // Si ya existe, marcamos como
duplicada
                break;
            }
        }

        // Si no existe, la añadimos ordenando por longitud y
alfabéticamente
        if (!ja_existeix) {
            bool afegida = false;
            for (list<string>::iterator it = paraules.begin(); it !=
paraules.end(); ++it) {
                if (p.length() < it->length() or (p.length() ==
it->length() and p < *it)) {
                    paraules.insert(it, p); // Insertamos en la posición
correcta
                    afegida = true;
                    break;
                }
            }

            // Si no hemos encontrado un lugar para insertarla, la
añadimos al final
            if (!afegida) {
                paraules.push_back(p);
            }
        }

        ++subconjunts; // Avanzar al siguiente subconjunto
    }
}

/* Pre: Cert

```

```

    Post: Retorna la llista de paraules que es poden formar usant 3 o més
lletres
    de la paraula s. La llista estarà ordenada per longitud i dins de
cada longitud
    en ordre alfabètic, sense duplicats. Llança un error si l'string s té
menys de tres lletres. */
void obte_paraules::obte_paraules(const string& s, const anagrames& A,
list<string>& paraules) throw(error) {
    // Inicializamos `paraules` como una lista vacía
    paraules = list<string>();

    // Validación de precondiciones
    if (s.size() < 3) {
        throw error(LongitudInvalida);
    }

    // Generar palabras para todas las longitudes posibles (de 3 a la
longitud de la palabra)
    for (nat k = 3; k <= s.size(); ++k) {
        list<string> resultats_parcials;

        // Obtener palabras de longitud k
        obte_paraules(k, s, A, resultats_parcials);

        // Insertar las palabras de manera manual, evitando duplicados
        while (!resultats_parcials.empty()) {
            string p = resultats_parcials.front(); // Extraer la primera
palabra
            resultats_parcials.pop_front(); // Eliminar la palabra de la
lista original

            // Comprobar si la palabra ya existe en la lista
            bool ja_existeix = false;
            for (list<string>::iterator it = paraules.begin(); it !=
paraules.end(); ++it) {
                if (*it == p) {
                    ja_existeix = true; // Si ya existe, marcamos como
duplicada
                    break;
                }
            }
        }
    }
}

```



```

        // Si no existe, la añadimos ordenando por longitud y
alfabéticamente
        if (!ja_existeix) {
            bool afegida = false;
            for (list<string>::iterator it = paraules.begin(); it !=
paraules.end(); ++it) {
                if (p.length() < it->length() or (p.length() ==
it->length() and p < *it)) {
                    paraules.insert(it, p); // Insertamos en la posición
correcta

                    afegida = true;
                    break;
                }
            }

            // Si no hemos encontrado un lugar para insertarla, la
añadimos al final
            if (!afegida) {
                paraules.push_back(p);
            }
        }
    }
}

```

VERSION 3 (OPTIMIZADA) :

```

#include "obte_paraules.hpp"
#include "iter_subset.hpp"
#include "word_toolkit.hpp"

/* Pre: 'paraules' està ordenada per longitud i, en cas d'igualtat, en
ordre lexicogràfic.
   Post: 'p' s'insereix a 'paraules' mantenint l'ordre si no hi és ja.
   Cost: O(n), on 'n' és el nombre d'elements de 'paraules'. */
void inserir_paraula_ordenada(const string& p, list<string>& paraules)
{ // cost O(n)
    list<string>::iterator it = paraules.begin();
    bool duplicat = false;
    bool trobat = false;

    // Recorre la llista per trobar la posició adequada on inserir la
paraula

```

```

while (it != paraules.end() and !trobat) {
    if (*it == p) {
        duplicat = true; // Marca que la paraula ja existeix a la
// llista
    }
    if (!duplicat and (p.length() < it->length() or (p.length() ==
it->length() and p < *it))) {
        trobat = true; // Determina la posició d'inserció si no és
duplicat
    } else {
        ++it;
    }
}

// Insereix la paraula només si no és duplicada
if (!duplicat) {
    paraules.insert(it, p);
}
}

/* Pre: Cert
Post: Retorna la llista de paraules que es poden formar usant k
lletres
de la paraula s. Llança error si k és major que la longitud de
l'string s o k < 3. */
void obte_paraules::obte_paraules(nat k, const string& s, const
anagrames& A, list<string>& paraules) throw(error) { // cost
O((kn)·(k+n))
    if (k > s.size() or k < 3) {
        throw error(LongitudInvalida); // Comprova si k és vàlid
    }

    paraules.clear(); // Neteja la llista de paraules
    iter_subset subconjunts(s.size(), k); // Genera subconjunts de
longitud k

    while (!subconjunts.end()) {
        string paraula_actual;
        // Construeix la paraula a partir del subconjunt actual
        for (nat i = 0; i < (*subconjunts).size(); ++i) {
            paraula_actual += s[(*subconjunts)[i] - 1];
        }
    }
}

```

```

        list<string> paraules_anagrames;
        // Obté les paraules que tenen el mateix anagrama canònic

A.mateix_anagrama_canonic(word_toolkit::anagrama_canonic(paraula_actual
), paraules_anagrames);

        // Insereix les paraules d'anagrama en ordre a la llista final
        for (list<string>::iterator it = paraules_anagrames.begin(); it !=
paraules_anagrames.end(); ++it) {
            inserir_paraula_ordenada(*it, paraules);
        }

        ++subconjunts; // Avança al següent subconjunt
    }
}

/* Pre: Cert
   Post: Retorna la llista de paraules que es poden formar usant 3 o més
lletres
   de la paraula s. La llista estarà ordenada per longitud i dins de cada
longitud
   en ordre alfabètic, sense duplicats. Llança un error si l'string s té
menys de tres lletres. */
void obte_paraules::obte_paraules(const string& s, const anagrames& A,
list<string>& paraules) throw(error) { //  $O(k=3\sum n(kn) \cdot (k+n)) \approx O(2n \cdot n)$ 
    if (s.size() < 3) {
        throw error(LongitudInvalida); // Comprova si la paraula té
almenys tres lletres
    }

    paraules.clear(); // Neteja la llista de paraules
    for (nat k = 3; k <= s.size(); ++k) { // Itera sobre totes les
longituds possibles de 3 fins a la mida de s
        list<string> resultats_parcials;
        obte_paraules(k, s, A, resultats_parcials); // Obté paraules de
longitud k

        // Insereix les paraules obtingudes en ordre a la llista final
        for (list<string>::iterator it = resultats_parcials.begin(); it !=
resultats_parcials.end(); ++it) {
            inserir_paraula_ordenada(*it, paraules);
        }
    }
}

```

}